

TAGS & CLI

```
<?php // a file of pure PHP: no ?> at the end
declare(strict_types=1); // first statement: strict
scalar types
echo "Hi\u{1d}"; ?> <?= $x ?> // short echo tag in
templates

php -v; php -m; php --ini # version, extensions, ini
files
php script.php arg1 # $argv[1] === 'arg1'
php -r "echo PHP_VERSION;" # one-liner
php -a # interactive shell
php -S localhost:8000 -t public # dev server only
php -l file.php # syntax check
```

TYPES & DECLARATIONS

Scalar `int float string bool` · **compound** `array object`
callable `iterable` · **null** `mixed void never` · **true** `false` (8.2) · **self** **parent** **static** · **enums** and **classes**.

```
function f(
    int|string $id, // union (8.0)
    Countable&Traversable $c, // intersection (8.1)
    (A&B)|null $dnf, // DNF (8.2)
    ?Book $book = null, // nullable: Book|null
): never { throw new Exception(); } // never returns
```

Without `strict_types`, "5" is coerced to `int`; with it, a `TypeError`. `get_debug_type($v)` names any value.

COMPARISONS (PHP 8 RULES)

<code>0 == "a"</code>	false (true before 8.0)
<code>"1" == "01"</code>	true: numeric strings
<code>"10" == "1e1"</code>	true
<code>100 == "1e2"</code>	true
<code>null == false</code>	true
<code>[] == false</code>	true
<code>"abc" == 0</code>	false since 8.0
<code>"1" === 1</code>	false: always use ===

Falsy: `false 0 0.0 "" "0" [] null (int)"12abc"` is 12; arithmetic on `"abc"` throws `TypeError` (8.0).

OPERATORS

```
$name = $input ?? 'guest'; // null coalescing
$oopts['mode'] ??= 'fast'; // assign if null/unset
$cmp = $a <=> $b; // -1, 0 or 1
$city = $user?->address?->city; // nullsafe (8.0)
$strlug = $title |> trim(...) |> strtolower(...); // pipe (8.5)
$a11 = [...$a, ...$b]; f(...$args); // spread
2 ** 10; intval(7, 2); 7 % 3; fdiv(1, 0); // 1024, 3, 1, INF
$$ = 'a' . 'b'; $x instanceof Book; @$silenced;
```

CONTROL FLOW

```
foreach ($books as $isbn => $book) { if ($book->late)
continue; }
for ($i = 0; $i < 3; $i++) {} while ($ok) {} do {}
while ($ok);
$label = match (true) { // strict ===, no fall-through
    $days === 0 => 'on time',
    $days < 7 => 'late',
    default => 'overdue', // else
};
UnhandledMatchError
};
switch ($x) { case 1: echo 'one'; break; default: echo
'?' ; }
require_once __DIR__ . '/config.php'; // include_once /
require_once
```

STRINGS

```
"Hi {user->name}, {a['k']}"; 'no $interpolation\u{1d}';
$sql = <<<SQL
    SELECT * FROM books WHERE id = ?
SQL; // heredoc; nowdoc: <<<'TXT'
str_contains($s, 'x'); str_starts_with($s, 'a');
str_ends_with($s, 'z');
strpos($s, 'a') !== false; // 0 is a valid position
sprintf('%05.2f|%-6s|%', 3.14159, 'ab', 42); // 03.14|ab
|42
explode('.', 'a,b'); implode(' ', $list); trim($s);
ucfirst($s);
mb_strlen('café'); mb_substr($s, 0, 3);
htmlspecialchars($s);
preg_match('/(?<year>d{4})/', $s, $m); $m['year'];
```

ARRAYS

```
$list = [1, 2, 3]; $map = ['a' => 1, 'b' => 2];
[$first, , $third] = $list; ['a' => $a] = $map; //
destructuring
array_map(fn($n) => $n * 2, $list);
array_filter($list, fn($n) => $n > 1);
array_reduce($list, fn($c, $n) => $c + $n, 0);
array_find($books, fn($b) => $b->late); // first match
(8.4)
array_any($list, fn($n) => $n > 2); array_all($list,
'is_int'); // 8.4
array_first($map); array_last($map); // values (8.5)
array_key_first($map); in_array(2, $list, true);
array_search(...);
array_column($rows, 'title', 'id'); array_combine($k,
$v);
usort($books, fn($a, $b) => $a->year <=> $b->year);
array_is_list($list); count($map); // sort rsort asort
ksort: in place
```

FUNCTIONS & CLOSURES

```
function fine(int $days, int $perDay = 25, int
...$extra): int
{ return $days * $perDay + array_sum($extra); }
fine($perDay: $0, days: 3); // named arguments (8.0)
function addTag(array &$tags, string $t): void { $tags[]
= $t; }
$rate = 25;
$f = function (int $d) use ($rate) { return $d * $rate;
}; // by value
$g = fn(int $d) => $d * $rate; // arrow fn: auto-
captures
$h = strlen(...); $m = $obj->method(...); // first-
class callable
static fn() => 1; // no $this binding
```

A MODERN CLASS

```
final class Book
{
    public const int MAX_LOANS = 3; // typed const (8.3)
    public private(set) int $loans = 0; // asymmetric
    (8.4)

    public function __construct(
        public readonly string $isbn, // promoted,
        private string $title,
    ) {}

    public string $display { // virtual hook (8.4)
        get => "{$this->title} ({this->isbn})";
    }

    public string $slug { set => strtolower($value); } //
set hook

    public function lend(): static { $this->loans++;
return $this; }
    public function withTitle(string $t): static
    { return clone($this, ['title' => $t]); } // clone
with (8.5)
    public static function fromArray(array $a): self
    { return new self($a['isbn'], $a['title']); }
}

readonly class (8.2) makes every property readonly. new
Book(...)->lend() needs no parentheses (8.4).
```

INTERFACES, TRAITS, ENUMS

```
interface Lendable { public function lend(): static; }
abstract class Item implements Lendable
{ abstract public function title(): string; }
trait Timestamps { public ?DateTimeImmutable $at = null;
}
class Dvd extends Item { use Timestamps;
#[Override] public function title(): string { return
'DVD'; }
    public function lend(): static { return $this; } }
enum Status: string { // backed enum (8.1)
    case Available = 'a'; case OnLoan = 'l';
    public function label(): string { return
ucfirst($this->name); }
    Status::from('a'); Status::tryFrom('x'); Status::cases();
}
```

MAGIC METHODS

<code>__construct</code>	<code>__destruct</code>	lifecycle		
<code>__get</code>	<code>__set</code>	<code>__isset</code>	<code>__unset</code>	inaccessible properties
<code>__call</code>	<code>__callStatic</code>	inaccessible methods		
<code>__toString</code>	string cast (Stringable)			
<code>__invoke</code>	<code>\$obj()</code>			
<code>__clone</code>	after <code>__clone</code>			
<code>__serialize</code>	<code>__unserialize</code>	serialisation (<code>__sleep</code> : 8.5 soft-deprecated)		
<code>__debugInfo</code>	<code>__set_state</code>	<code>var_dump</code> , <code>var_export</code>		

ATTRIBUTES

```
#[Override] 8.3 · #[Deprecated] 8.4 · #[NoDiscard] 8.5 ·
#[SensitiveParameter] 8.2 · #[AllowDynamicProperties]
8.2 · #[ReturnTypeWillChange] 8.1 ·
#[Attribute(Attribute::TARGET_CLASS)] declares one.
Read with (new ReflectionClass(X::class))-
>getAttributes().
```

EXCEPTIONS

Throwable → **Error** (`TypeError`, `ValueError`,
`ArithmeticError`/`DivisionByZeroError`,
`UnhandledMatchError`) and **Exception** (`RuntimeException`,
`LogicException`, `InvalidArgumentException`, `JsonException`,
...).

```
try { $loan->close(); }
catch (NotFound|Expired $e) { $logger->warning($e-
getMessage()); }
catch (Throwable $e) { throw new LoanError('failed',
previous: $e); }
finally { $lock->release(); } // return in finally: 8.6
deprecated
throw new InvalidArgumentException("bad isbn"); // also
an expression
```

GENERATORS & FIBERS

```
function rows(string $file): Generator {
    $h = fopen($file, 'r');
    while (($r = fgets($h, escape: '')) !== false) {
yield $r; }
    } // lazy: one row in memory
function pairs(): Generator { $got = yield 'k' => 1;
yield from [2, 3]; }
$gen = pairs(); $gen->key(); $gen->send('x'); // resume
with a value
$fiber = new Fiber(function (): string {
    $in = Fiber::suspend('paused'); // cooperative, not
threads
    return "got $in";
});
$fiber->start(); $fiber->resume('data'); $fiber-
>getReturn();
```

NAMESPACES

```
namespace Library\Catalogue; // file
src/Catalogue/Book.php
use Library\Loans\{Loan, Fine}; // group use
use function Library\Support\slugify;
use const Library\VERSION;
\strlen($s); Book::class; // 'Library\Catalogue\Book'
```

DATES, JSON, FILES

```
$due = new DateTimeImmutable('2026-10-05', new
DateTimeZone('UTC'));
$due->modify('+14 days')->format('Y-m-d'); // immutable:
new object
$due->diff($now)->days;
DateTimeImmutable::createFromFormat('d/m/Y', $s);
json_encode($data, JSON_THROW_ON_ERROR |
JSON_PRETTY_PRINT);
json_decode($json, true, flags: JSON_THROW_ON_ERROR);
json_validate($j);
file_put_contents($f, $s, LOCK_EX);
file_get_contents($f);
foreach (new SplFileObject($f) as $line) {} is_file($f);
unlink($f);
```

PASSWORDS & CRYPTO

```
$hash = password_hash($pw, PASSWORD_DEFAULT); // bcrypt,
cost 12 (8.4)
password_verify($pw, $hash);
password_needs_rehash($hash, PASSWORD_DEFAULT);
$token = bin2hex(random_bytes(32)); random_int(1, 6);
hash_equals($expected, $given); hash_hmac('sha256',
$msg, $key);
$key = sodium_crypto_secretbox_keygen(); $n =
random_bytes(24);
$box = sodium_crypto_secretbox($msg, $n, $key);

Never md5/sha1 for passwords; never hard-code keys (load
them from the environment).
```

COMPOSER

```
composer init | require monolog/monolog | require --dev
pestphp/pest
composer install # from composer.lock (commit it)
composer update vendor/pkg; composer audit; composer
outdated
composer dump-autoload -o; composer run test

"require": { "php": ">=8.4" },
"autoload": { "psr-4": { "Library\\": "src/" } },
"autoload-dev": { "psr-4": { "Tests\\": "tests/" } }
```

PSRS

1 basic style · **3** logger · **4** autoload · **6/16** cache · **7** HTTP
messages · **11** container · **12**→**PER CS 3** style · **13** links ·
14 events · **15** middleware · **17** HTTP factories · **18** HTTP
client · **20** clock. PSR-0 and PSR-2 are deprecated.

PHPUNIT & PEST

```
final class FineTest extends TestCase {
    #[Test,DataProvider('days')]
    public function fines(int $d, int $c): void
    { $this->assertSame($c, (new Fines())->for($d)); }
    public static function days(): array { return [[1,
25], [2, 50]]; }
}
it('caps fines', fn() => expect((new Fines())->for(99))-
>toBe(100));

vendor/bin/phpunit --testdox; vendor/bin/pest --coverage
--mutate
```

CHANGED SINCE OLDER TUTORIALS

each() removed 8.0 → foreach create_function() removed 8.0 → closures mysql_* removed 7.0 → PDO/mysqli

mcrypt removed 7.2 → sodium 0 == "a" true false since 8.0 FILTER_SANITIZE_STRING deprecated 8.1 → htmlspecialchars strftime() deprecated 8.1 → Intl/DateFormatter

dynamic properties deprecated 8.2 → declare then __sleep/_wakeup soft-deprecated 8.5 → __serialize string \$s = null deprecated 8.4 → ?string

'\s' backticks deprecated 8.5 (integer) casts deprecated 8.5 → (int)